

4. Step 2 — Exploratory Data Analysis (EDA)

EDA helps us understand the relationship between advertising spend and sales, identify outliers, and select the relevant features.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Correlation heatmap
plt.figure(figsize=(8, 5))
sns.heatmap(df.corr(), annot=True, cmap='Blues', fmt='.2f')
plt.title('Feature Correlation Matrix')
plt.tight_layout()
plt.savefig('correlation_heatmap.png')
plt.show()

# Sales vs TV spend scatter
plt.figure(figsize=(6, 4))
plt.scatter(df['TV'], df['Sales'], alpha=0.6, color='steelblue')
plt.xlabel('TV Advertising Budget ($000s)')
plt.ylabel('Sales (Units)')
plt.title('TV Spend vs Sales')
plt.tight_layout()
plt.savefig('tv_vs_sales.png')
plt.show()

# Distribution of Sales
plt.figure(figsize=(6, 4))
df['Sales'].hist(bins=20, color='coral', edgecolor='black')
plt.title('Sales Distribution')
plt.xlabel('Sales'); plt.ylabel('Frequency')
plt.tight_layout()
plt.savefig('sales_distribution.png')
plt.show()

print("Key Correlations:")
print(df.corr()['Sales'].sort_values(ascending=False))
```

5. Step 3 — Model Building (Regression)

We build and compare three regression models: **Linear Regression**, **Random Forest Regressor**, and **Gradient Boosting Regressor**. The dataset is split 80/20 for training and testing.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Train-test split (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42)

# Model 1: Linear Regression
lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

# Model 2: Random Forest
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
```

```
y_pred_rf = rf.predict(X_test)

# Model 3: Gradient Boosting
gb = GradientBoostingRegressor(n_estimators=100, learning_rate=0.1,
                               max_depth=3, random_state=42)
gb.fit(X_train, y_train)
y_pred_gb = gb.predict(X_test)

# Evaluation Function
def evaluate(name, y_true, y_pred):
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)
    print(f'{name:30s} | MAE: {mae:.2f} | RMSE: {rmse:.2f} | R2: {r2:.4f}')

evaluate("Linear Regression", y_test, y_pred_lr)
evaluate("Random Forest", y_test, y_pred_rf)
evaluate("Gradient Boosting", y_test, y_pred_gb)
```